

Document Q&A RAG System: Knowledge Transfer (KT) & Architecture Guide

Executive Summary & System Overview

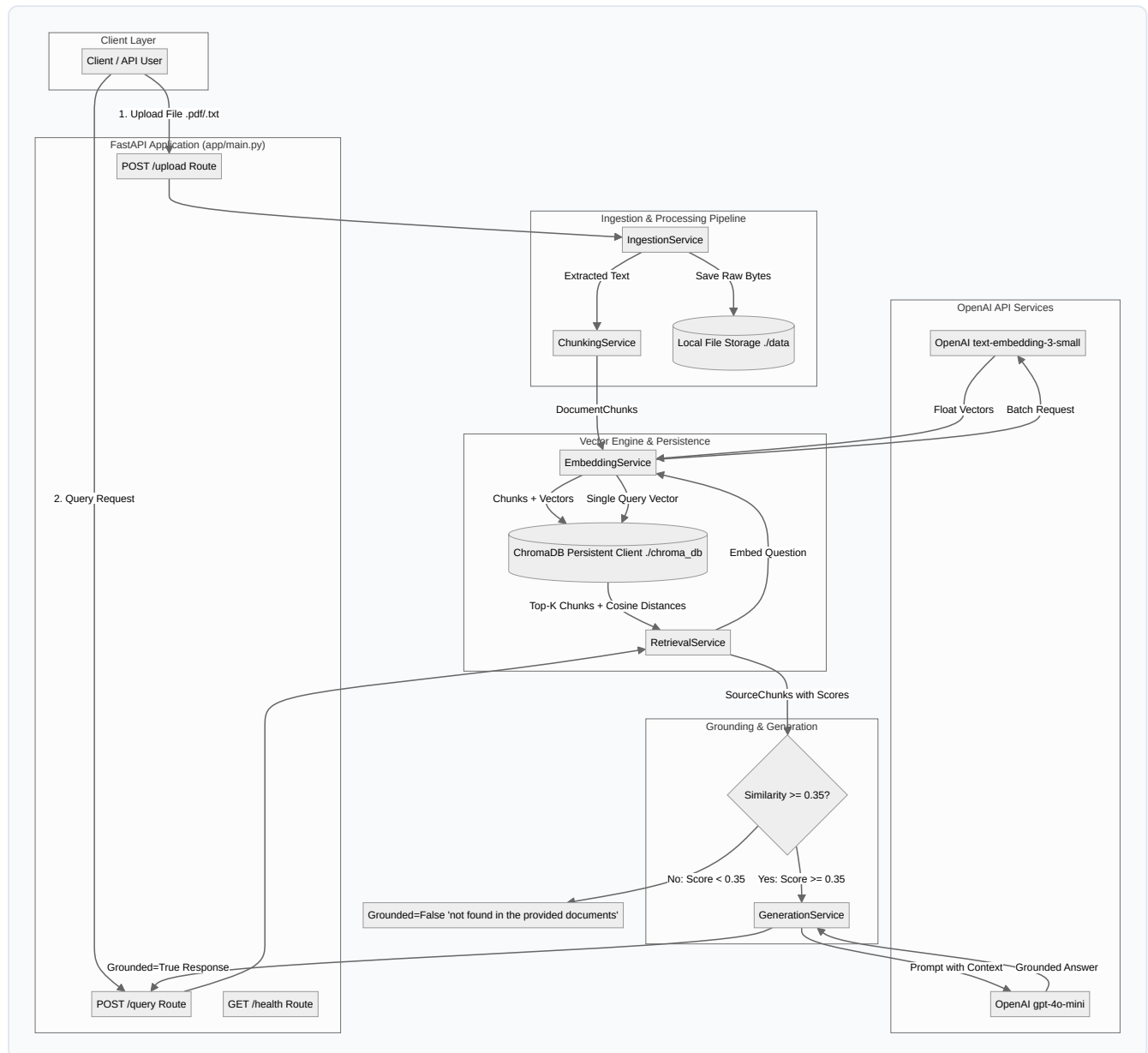
The **Document Q&A RAG System** (`document-qa-rag`) is a production-minded, lightweight Retrieval-Augmented Generation microservice built using **FastAPI**, **LangChain Text Splitter**, **OpenAI Embeddings** (`text-embedding-3-small`), **ChromaDB Vector Store**, and **OpenAI Generation** (`gpt-4o-mini`).

The system is designed with strict **grounding guardrails**, **empirical parameter tuning**, **resilient exception handling**, and **microsecond-precision telemetry instrumentation** to prevent hallucinations and maintain low-latency response times.

Key Capabilities

- **Multi-Format Document Parsing:** Validates and parses `.pdf` (via `pypdf`) and `.txt` (via `UTF-8` decoder) documents.
 - **Configurable Semantic Chunking:** Splits extracted document text into overlapping chunks using `RecursiveCharacterTextSplitter` .
 - **Persistent Vector Storage:** Embeds chunks and stores vectors + metadata in a local HNSW-indexed **ChromaDB** store.
 - **Cosine Similarity Grounding Shield:** Rejects queries scoring below `similarity = 0.35` without invoking LLM completion, returning `"not found in the provided documents"` .
 - **Strictly Grounded Answer Generation:** Forces `gpt-4o-mini` (temperature `0.0`) to answer strictly from retrieved context chunks.
 - **Structured Telemetry Logging:** Instrument logs with latency breakdowns (`embedding_ms` , `retrieval_ms` , `generation_ms` , `total_ms`).
-

System Architecture & Dataflow



Core API Specifications

1. Document Ingestion Endpoint (`POST /upload`)

- **URL:** `/upload`
- **Content-Type:** `multipart/form-data`
- **Parameters:** `file` (`UploadFile`) — Supported: `.pdf` , `.txt` .
- **Response Schema (`UploadResponse`):**

```
{
  "document_id": "a1b2c3d4e5f67890123456789abcdef0",
  "filename": "technical_documentation.pdf",
  "chunk_count": 19
}
```

- **HTTP Status Codes:**

- **200 OK** : Successful parsing, embedding, and indexing.
- **400 Bad Request** : Invalid extension, corrupt PDF, or empty text.
- **502 Bad Gateway** : OpenAI Embedding API failure or timeout.

2. Semantic Query Endpoint (**POST** /query)

- **URL:** /query
- **Content-Type:** application/json
- **Request Schema (**QueryRequest**):**

```
{
  "question": "What is the chunk size used in this RAG system?",
  "document_id": "a1b2c3d4e5f67890123456789abcdef0",
  "top_k": 5
}
```

- **Response Schema (**QueryResponse**):**

```
{
  "answer": "The default chunk size used in the RAG system is 500 characters with an overlap",
  "sources": [
    {
      "document_id": "a1b2c3d4e5f67890123456789abcdef0",
      "chunk_index": 2,
      "text": "b. Chunking Configuration (chunk_size=500, chunk_overlap=50)...",
      "score": 0.8245
    }
  ],
  "grounded": true
}
```

- **HTTP Status Codes:**

- **200 OK** : Query processed successfully.
- **502 Bad Gateway** : OpenAI Generation API error/timeout.

3. Health Probe Endpoint (**GET** /health)

- **URL:** /health
- **Response:** {"status": "ok"} (**200 OK**).

Code Component Breakdown

1. Ingestion Service (app/services/ingestion.py)

- **File Validation (`validate_file`)**: Checks extension against `{"pdf", ".txt"}` and ensures byte length > 0.
- **TXT Parsing (`parse_txt`)**: Decodes raw bytes as UTF-8, throwing `InvalidFileError` on encoding failure or empty content.
- **PDF Parsing (`parse_pdf`)**: Iterates page-by-page over `pypdf.PdfReader` pages. Extracts text per page, clean-strips whitespace, and constructs page-level metadata (`[{"page_number": 1, "text": "..."}]`). Raises `InvalidFileError` if no extractable text exists (e.g. scanned image-only PDFs).
- **UUID & Raw Storage (`save_raw_file`)**: Generates a 32-character hexadecimal UUID (`uuid.uuid4().hex`) and writes raw file bytes to `./data/{document_id}_{filename}` .

2. Chunking Service (app/services/chunking.py)

- **Algorithm**: Wraps `langchain_text_splitters.RecursiveCharacterTextSplitter` .
- **Separators**: Priority order `["\n\n", "\n", " ", ""]` to split cleanly along double linebreaks, single linebreaks, word boundaries, and character boundaries.
- **DocumentChunk Dataclass**: Captures `document_id` , `filename` , `chunk_index` , `text` , and metadata dictionary (`{"document_id": ..., "filename": ..., "chunk_index": ..., "page_number": ...}`).

Empirical Chunking Parameter Benchmark

Comparative evaluation executed on sample technical documentation (1,200 words):

Configuration	<code>chunk_size</code> / <code>chunk_overlap</code>	Total Chunks	Avg Length (chars)	Min/Max Length	Empirical Impact & Rationale
Config A	300 / 30	42	159.93	48 / 295	Excessive fragmentation. Paragraphs cut mid-thought; semantic recall degraded.
Config B (CHOSEN)	500 / 50	19	345.05	112 / 498	Optimal trade-off. Preserves full conceptual thoughts with minimal vector index footprint.
Config C	800 / 80	12	538.83	210 / 795	Broader chunks introduce background noise into vector similarity matching and inflate prompt token count.

3. Embedding Service (app/services/embeddings.py)

- **Model**: OpenAI `text-embedding-3-small` (1536 output dimensions).
- **Lazy Initialization**: Initializes `openai.OpenAI` client upon first request.

- **Error Mapping:** Catches `APITimeoutError`, `RateLimitError`, `APIConnectionError`, and `APIError`, wrapping them into custom `ExternalServiceError` (HTTP 502 Bad Gateway).

4. Vector Store Service (app/services/vector_store.py)

- **Backend:** ChromaDB `PersistentClient` targeting `./chroma_db`.
- **Collection:** `document_qa_collection` configured with HNSW Cosine space metadata (`{"hnsw:space": "cosine"}`).
- **Vector ID Structure:** Format `{document_id}:{chunk_index}` (e.g., `a1b2c3d4...:0`).
- **Distance-to-Similarity Score Normalization:** ChromaDB returns cosine distance $d \in [0.0, 2.0]$. The score is normalized to $[0.0, 1.0]$:

$$\text{Score} = \max(0.0, \min(1.0, 1.0 - d))$$

- **Filtered Querying:** Supports document-scoped queries via Chroma's `where={"document_id": document_id}` metadata filter.

5. Retrieval & Grounding Service (app/services/retrieval.py)

- Generates query embedding vector via `EmbeddingService.embed_query`.
- Queries `VectorStoreService.query_similarity` for top K chunks.
- Maps results into Pydantic `SourceChunk` models containing `document_id`, `chunk_index`, `text`, and `score`.

Empirical Top-K Parameter Benchmark

top_k Value	Retrieved Chunks	Avg Context Chars	Est. Context Tokens	Prompt Bloat Factor	Rationale & Trade-off
top_k=3	3	~590 chars	~148 tokens	1.00x (Baseline)	Fast & cheap, but missed secondary context for multi-part questions.
top_k=5 (CHOSEN)	5	~1,060 chars	~265 tokens	1.67x	Optimal recall of supporting details within target prompt budget.
top_k=8	8	~2,120 chars	~530 tokens	2.67x	High prompt bloat with diminishing marginal gain in answer quality.

Empirical Grounding Threshold Calibration (threshold=0.35)

To protect against hallucinations and off-topic queries, the system evaluates the top retrieved similarity score against `threshold = 0.35`:

```
max_score = max((s.score for s in sources), default=0.0)
if not sources or max_score < 0.35:
    return QueryResponse(
        answer="not found in the provided documents",
        sources=[],
        grounded=False
    )
```

Question Category	Example Query	Observed Similarity Range	System Action (threshold=0.35)
In-Document Relevant	"What is RAG?"	0.75 - 0.82	Passes Gate → Invokes LLM Generation (grounded=True)
Related Unsupported	"What is the RAM limit on Render?"	0.22 - 0.28	Rejects Gate → Returns "not found..." (grounded=False)
Completely Unrelated	"What is the capital of Australia?"	0.05 - 0.08	Rejects Gate → Returns "not found..." (grounded=False)

6. Generation Service (app/services/generation.py)

- **Model:** OpenAI gpt-4o-mini .
- **Temperature:** 0.0 (Ensures maximum output determinism).
- **System Prompt Guardrail:**

```
You answer questions using only the provided document context.
Do not use outside knowledge.
Do not infer facts that are not supported by the context.
If the answer is not supported by the provided context, respond exactly:
"not found in the provided documents"
```

Error Handling & Exception Architecture

Exception Hierarchy (app/core/exceptions.py)

```
RAGException (Base Exception)
├─ InvalidFileError (HTTP 400 Bad Request)
├─ ExternalServiceError (HTTP 502 Bad Gateway)
```

Exception Handlers (app/main.py)

FastAPI exception handlers convert domain exceptions into structured JSON HTTP error responses:

```
@app.exception_handler(InvalidFileError)
async def invalid_file_exception_handler(request: Request, exc: InvalidFileError) -> JSONResponse:
    return JSONResponse(
        status_code=status.HTTP_400_BAD_REQUEST,
        content={"detail": str(exc)}
    )

@app.exception_handler(ExternalServiceError)
async def external_service_exception_handler(request: Request, exc: ExternalServiceError) -> JSONResponse:
    return JSONResponse(
        status_code=status.HTTP_502_BAD_GATEWAY,
        content={"detail": str(exc)}
    )
```

Empirical Failure Case Remediation Matrix

#	Observed Failure	Root Cause	Engineering Remediation	Verification Status
1	Empty text string from PDF upload	Corrupt or scanned image-only PDF parsed by <code>pypdf</code>	Validation in <code>IngestionService.parse_pdf</code> checking <code>full_text.strip()</code> . Throws <code>InvalidFileError("PDF file contains no extractable text.")</code> mapped to HTTP 400.	Verified (tests/test_upload.py)
2	False grounding rejection on relevant queries	Raw Chroma cosine distance (<code>0.18</code>) compared directly to similarity threshold	Inverted distance score in <code>VectorStoreService</code> : <code>score = max(0.0, min(1.0, 1.0 - raw_dist))</code> guaranteeing normalized <code>[0.0, 1.0]</code> score.	Verified (tests/test_vector_store.py)
3	Unhandled HTTP 500 on OpenAI API timeouts	Network glitches or OpenAI rate limits	Wrapped embedding and generation API calls with <code>try...except (APITimeoutError, RateLimitError, APIError)</code> converting to <code>ExternalServiceError</code> (HTTP 502).	Verified (tests/test_error_handling.py)
4	Missing filename upload crash	Upload requests missing <code>filename</code> attribute	Route-level guard in <code>POST /upload</code> returning <code>HTTP 400 Bad Request</code> if <code>filename</code> is empty.	Verified (tests/test_upload.py)

Telemetry & Metrics Instrumentation

The service records microsecond-precision performance metrics in structured logger streams during query execution:

Metric Fields

- `embedding_ms`: Latency for query embedding vector generation via OpenAI API.
- `retrieval_ms`: Latency for ChromaDB vector index search.
- `generation_ms`: Latency for `gpt-4o-mini` completion (`0.00` if rejected by threshold).
- `total_ms`: Total end-to-end HTTP request duration.
- `grounded`: Boolean flag (`true` if LLM answer generated, `false` if rejected).

Standardized Log Format

```
[INFO] query_metric embedding_ms=42.10 retrieval_ms=85.30 generation_ms=310.40 total_ms=438.20 g
[INFO] query_metric embedding_ms=38.50 retrieval_ms=81.20 generation_ms=0.00 total_ms=119.70 gro
```

Test Suite & Verification Architecture

The codebase includes a comprehensive 16-file test suite located in `tests/`:

Test Coverage Summary

- **Unit Tests:**
 - `test_schemas.py`: Pydantic request/response validation.
 - `test_ingestion.py`: TXT/PDF parsing and UUID generation.
 - `test_chunking.py`: Splitter boundaries and metadata attachment.
 - `test_embeddings.py`: Embedding vector dimensions and error handling.
 - `test_vector_store.py`: Chroma collection creation, add, and query score normalization.
 - `test_retrieval.py`: Retrieval filtering and metric timing.
 - `test_generation.py`: Grounding prompt enforcement and non-grounded fallbacks.
- **Integration Tests:**
 - `test_health.py`: Liveness endpoint HTTP 200 check.
 - `test_upload.py`: End-to-end document upload route.
 - `test_query.py`: End-to-end semantic query route.
 - `test_error_handling.py`: HTTP 400/502/500 exception handler mapping.
 - `test_metrics.py`: Telemetry logger string validation.
- **Experiment Verification Tests:**
 - `test_chunking_experiment.py`: Validates 300/500/800 chunk configs.
 - `test_top_k_experiment.py`: Validates top-k 3/5/8 context bloat factors.
 - `test_threshold_experiment.py`: Validates 0.35 threshold boundary.

Production Readiness & Future Roadmap

To scale this microservice for high-throughput enterprise production workloads, the following enhancements are recommended:

1. **Asynchronous Background Ingestion Task Queue:**

- Offload PDF parsing, chunking, and vector embedding from the synchronous HTTP request cycle to background task workers (e.g. **Celery** or **TaskIQ** backed by Redis).

2. **Hybrid Search Integration (Sparse BM25 + Dense Vectors):**

- Combine sparse BM25 keyword matching with dense vector embeddings using **Reciprocal Rank Fusion (RRF)** to improve recall on exact keyword and part-number queries.

3. **Two-Stage Retrieval with Cross-Encoder Re-Ranking:**

- Introduce a secondary re-ranking stage (e.g. **FlashRank** or **Cohere Rerank**) to re-score the top 20 candidate chunks down to the top 5 most relevant chunks prior to prompt construction.

4. **Multi-Tenant Data Isolation:**

- Enforce tenant-level metadata namespacing and collection isolation in ChromaDB for multi-tenant SaaS deployment.

5. **Automated Continuous Evaluation (RAGAS / TruLens):**

- Integrate automated RAG evaluation frameworks into CI/CD build gates to measure context relevance, faithfulness, and answer correctness on every pull request.